

Assignment 1: MVC Design Pattern and Its Variants

The Model-View-Controller (MVC) design pattern is a widely used architectural pattern that separates an application into three interconnected components. This separation improves maintainability, scalability, and clarity of software systems.

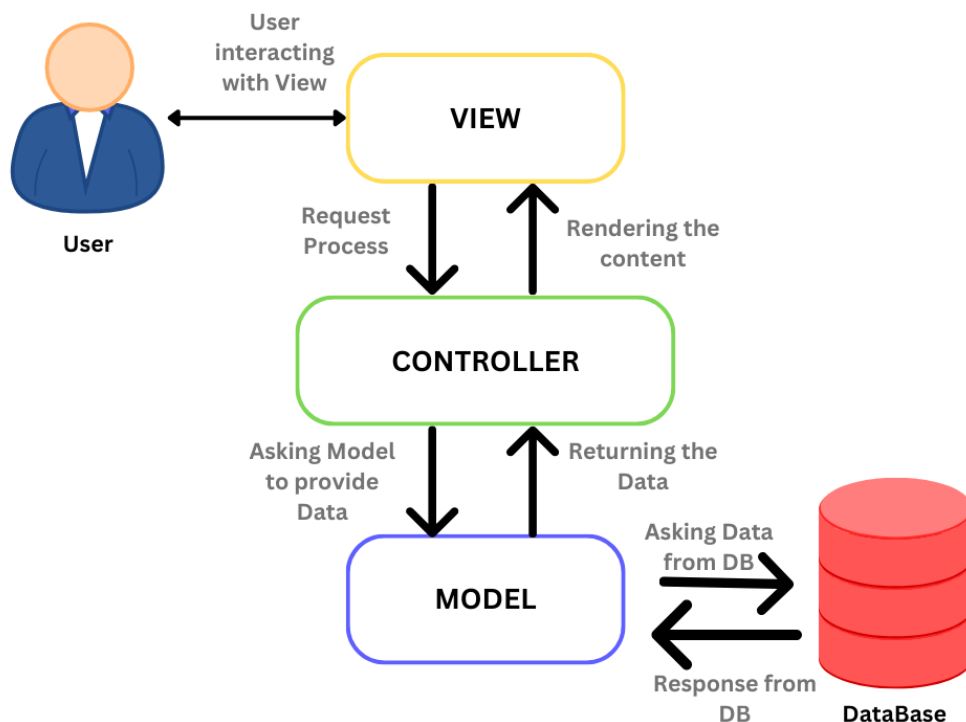
1. MVC (Model-View-Controller)

MVC divides an application into three parts:

- Model: Contains application data and business logic.
- View: Displays the data to the user.
- Controller: Handles user input and updates the Model and View accordingly.

In MVC, the Controller acts as an intermediary between the View and the Model.

Diagram: MVC Structure



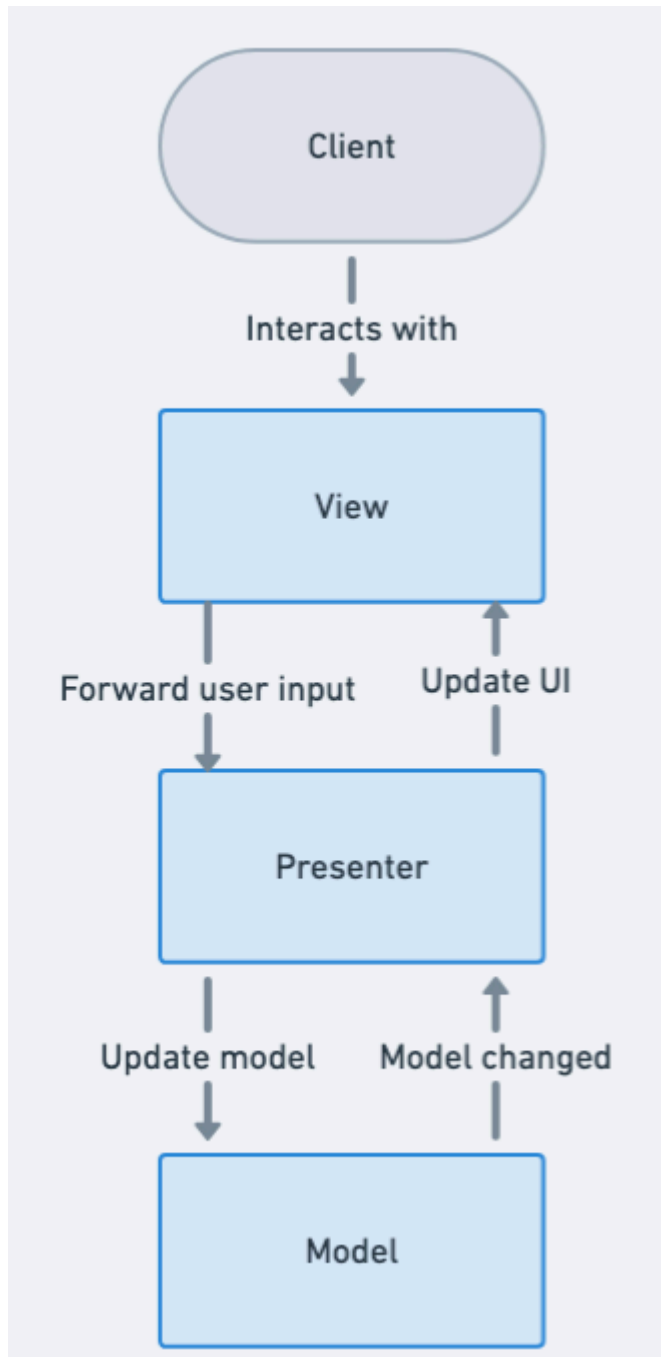
Appropriate Usage:

MVC is best suited for applications with complex interfaces and clear separation between business logic and presentation, such as web and enterprise applications.

2. MVP (Model-View-Presenter)

MVP is a variation of MVC where the Presenter replaces the Controller. The Presenter contains all presentation logic and directly updates the View. The View remains passive, improving testability.

Diagram: MVP Structure



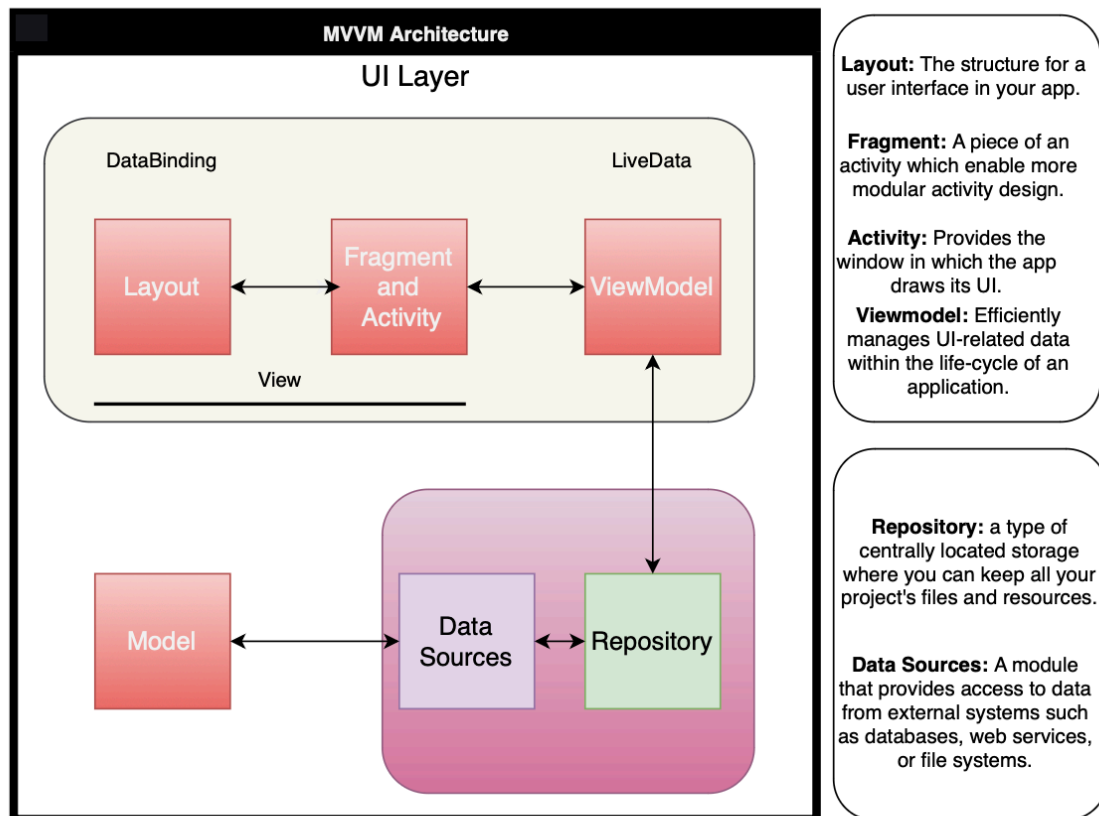
Appropriate Usage:

MVP is suitable for applications that require extensive unit testing and have complex UI logic, such as desktop applications.

3. MVVM (Model-View-ViewModel)

MVVM introduces a ViewModel that acts as an abstraction of the View. The ViewModel uses data binding to automatically synchronize the View and Model, reducing manual UI updates.

Diagram: MVVM Structure



Appropriate Usage:

MVVM is ideal for frameworks supporting data binding, such as Angular, React, and WPF-based applications.

Conclusion

MVC and its variants address different application needs. MVC emphasizes separation of concerns, MVP focuses on testability, and MVVM simplifies UI development through data binding. Choosing the right variant depends on application complexity and framework support.